

INFO0027 - File Deduplication Service

Pavlov Aleksandr (s2400691)

1 Architecture and Design

The service has four layers:

- **Bootstrap** - `FileDeduplicationBootstrapImpl` is the composition root, discovered via `ServiceLoader`. It initialises all engines and wires them together.
- **Frontend Gate** - `FrontendGateImpl` exposes the two-method JSON API required by the frontend team (`accept` / `acceptStream`).
- **Deduplication engines** - `KotlinFileDeduplicator` (exact, SHA-256 + byte comparison) and `ImageFileDeduplicator` (similarity, OpenCV template matching) both implement the internal `FileDeduplicator` interface.
- **Storage integration** - `FileDeduplicator` extends the provided `StorageChecker` interface.

On startup, `initialize` builds an index of the entire `VirtualFileSystem`.

At request time, `FrontendGateImpl` selects the scan type and returns grouped results as JSON.

The Storage Team uses `findDuplicate(path)`. It first checks the existing index; if the entry is missing, it computes the hash.

1.1 UML Class Diagram

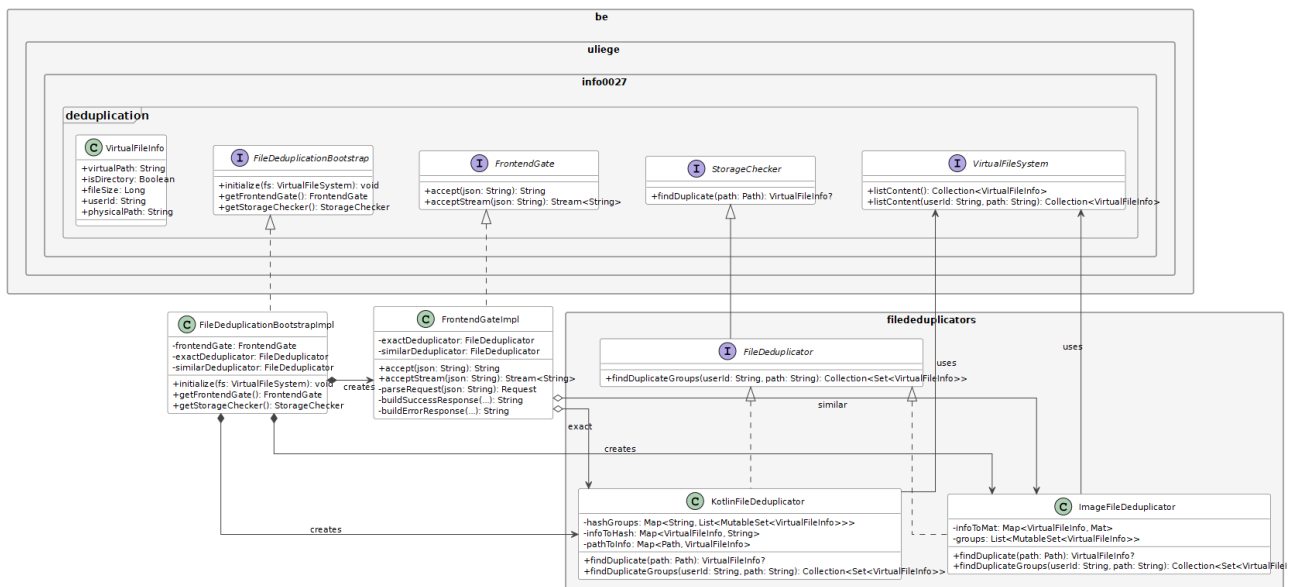


Figure 1: UML Class Diagram.

2 Software Design Patterns

2.1 Strategy

`FileDeduplicator` is the main interface. `KotlinFileDeduplicator` and `ImageFileDeduplicator` are different implementations. `FrontendGateImpl` chooses which one to use at runtime based on `scan_type`.

To add a new deduplication engine, only a new class is needed. The routing code does not need to change.

Trade-off: Both engines are created at startup, even if only one is used. This uses more memory. Lazy initialization would reduce startup cost, but the first request would be slower.

2.2 Adapter

`FrontendGateImpl` acts as an adapter between external JSON requests and the internal deduplication system. It converts JSON input into calls to `FileDeduplicator` implementations and converts internal results back into JSON responses. It also catches `RequestExceptions` and returns `{"status": "error"}` responses instead of raw exceptions.

This allows the internal deduplication logic to stay independent from the external request format.

Trade-off: `FrontendGateImpl` depends on both the external API format and the internal deduplication interfaces. Changes on either side may require updating the adapter.

2.3 Service Locator

`FileDeduplicationBootstrapImpl` acts as a service locator. It creates and stores instances of `FrontendGate` and `StorageChecker`. It then provides access to them through getter methods.

The class is also registered in `META-INF/services/`, which allows the test suite to find it automatically using `FileDeduplicationBootstrap` without knowing the implementation class name.

3 Quality Assurance

3.1 Functional Suitability - Functional Correctness

Application: `findDuplicateGroups` must return groups of size ≥ 2 scoped to the requested user and path. `KotlinFileDeduplicator` confirms equality via SHA-256 and bitwise comparison to rule out hash collisions. `FrontendGateImpl` ensures every response is a valid JSON object with a `groups` array on success or a `message` on error.

Tracking: Automated unit tests.

Ensured by: `KotlinFileDeduplicatorTest` verifies group count, path scoping, and user isolation. `FrontendGateImplTest` verifies JSON structure and stream cardinality.

3.2 Maintainability - Modularity

Application: All deduplication logic is behind `FileDeduplicator`; `FrontendGateImpl` never imports engine or OpenCV types directly. Swapping to a C-backed engine or a different JSON library touches one class only. Extension functions (`VirtualFileSystemExtensions`, `PathExtensions`) add behaviour to external types, keeping engine classes clean.

Tracking: Enforced by code review: any new engine must implement `FileDeduplicator` and be wired only in `FileDeduplicationBootstrapImpl`.

Ensured by: Interface-based design and the Strategy pattern prevent direct coupling between the routing layer and concrete implementations.

3.3 Reliability - Fault Tolerance

Application: `FrontendGateImpl` catches exceptions and returns a well-formed error response, so callers never receive a raw exception. `ImageFileDeduplicator` silently skips non-image files when OpenCV returns an empty `Mat`, so one bad file cannot abort the entire indexing pass.

Tracking: Automated tests.

Ensured by: `FrontendGateImplTest` covers invalid input combinations. `ImageFileDeduplicatorTest` verifies that non-image files are skipped without error.

4 Test Plan

Tests are run via `./gradlew test`. The full implementation is in `src/test/kotlin/`. The fixture `TestFileSystem` simulates two users (alice, bob) with controlled file contents and sample images to produce known duplicate groups. Test logs are provided in the annex.

- **KotlinFileDeduplicatorTest** - covers group correctness, path scoping, user isolation, and on-the-fly duplicate detection via `findDuplicate` for both indexed and new files.
- **ImageFileDeduplicatorTest** - covers similarity-based grouping, cross-user detection, and correct handling of non-image files.
- **FrontendGateImplTest** - covers JSON parsing, all invalid input combinations, and stream output correctness.

5 Design Process

Transitive image grouping. If $A \sim B$ and $B \sim C$, all three must share a group. The indexing loop adds each new image to the first existing group containing a similar member, otherwise it opens a new group.

Hash-collision safety. SHA-256 collisions are infeasible in practice, but files within the same hash bucket are still partitioned by bitwise equality to avoid false positives.

Serving two teams with one object. `FileDeduplicator` extends `StorageChecker`, so the same engine instance satisfies both the Frontend and Storage contracts.

Extending the provided VFS interface. The provided `VirtualFileSystem` interface only lists direct children of a given path, it does not offer recursive traversal or a way to retrieve all files across all users. Rather than creating a wrapper class or subclass, extension functions were added to `VirtualFileSystem`.

6 Limitations

- **$O(n)$ image lookup.** `findDuplicate` for images scans all stored matrices, which may be too slow for on-the-fly upload checking at scale.
- **Transitive similarity is not guaranteed.** With a long chain of slightly changing images, the first and last images could end up being totally different.
- **Duplicate memory index.** Each deduplication engine keeps its own file index, which increases memory usage.

Declaration on Generative AI

During the preparation of this work, the author(s) used Claude (Anthropic) to: review and proofread the report, reformulate certain sections for improved readability, assist in the creation of the UML diagram, generate some test cases, and generate documentation comments for some functions. After

using these tools, the author(s) reviewed and edited the content as needed and take(s) full responsibility for the publication's content.

A Test Logs

Output of `./gradlew test` (Gradle 9.4.0, JDK 25, Kotlin 2.3.20).

> Task :test

```
KotlinFileDeduplicatorTest > alice has one duplicate group across root PASSED
KotlinFileDeduplicatorTest > alice docs folder has no duplicates PASSED
KotlinFileDeduplicatorTest > bob has no duplicates within his files PASSED
KotlinFileDeduplicatorTest > findDuplicate finds duplicate for indexed file PASSED
KotlinFileDeduplicatorTest > findDuplicate returns null for unique indexed file PASSED
KotlinFileDeduplicatorTest > findDuplicate finds duplicate for new file not in index PASSED
KotlinFileDeduplicatorTest > findDuplicate returns null for new unique file PASSED
```

```
ImageFileDeduplicatorTest > alice has one duplicate image group PASSED
ImageFileDeduplicatorTest > bob has no duplicate image groups PASSED
ImageFileDeduplicatorTest > findDuplicate finds similar image from another user PASSED
ImageFileDeduplicatorTest > findDuplicate finds similar image within same user PASSED
ImageFileDeduplicatorTest > findDuplicate returns null for non-image file PASSED
```

```
FrontendGateImplTest > accept returns success for valid exact scan PASSED
FrontendGateImplTest > accept returns success for valid similar scan PASSED
FrontendGateImplTest > accept returns error for invalid json PASSED
FrontendGateImplTest > accept returns error for missing action PASSED
FrontendGateImplTest > accept returns error for unsupported action PASSED
FrontendGateImplTest > accept returns error for unsupported scan_type PASSED
FrontendGateImplTest > accept returns error for missing path PASSED
FrontendGateImplTest > accept returns error for missing user PASSED
FrontendGateImplTest > acceptStream returns one element per duplicate group PASSED
FrontendGateImplTest > acceptStream returns error stream on invalid request PASSED
```

BUILD SUCCESSFUL in 14s